



CEP REPORT

DATA STRUCTURE AND ALGORITHMS

Course Code: CS-218

Submitted by:

Syed Irfan Raza CS-24118

M. Taha Siddiqui CS-24122

M. Abdullah Hamid CS-24125

Syed Shaheer Sohail CS-24135

Submitted to:

Miss Urooj

Problem Description

The objective of this project is to design and implement a file compression tool that reduces the size of text files without losing any data. The tool must include an intuitive graphical user interface (GUI) to allow users to easily select files for compression and decompression. A suitable compression algorithm such as Huffman Coding should be used to efficiently encode and decode file contents.

Distinguishing Features of the Project

Huffman Coding Implementation — character-based frequency encoding

Graphical User Interface (Tkinter) — no command-line required

Custom .huf file format — structured encoded data

Reversible Compression — guaranteed lossless recovery

Binary Encoding + Padding Logic

Compression Stats — size and ratio shown

Individual Contributions

Core Compression Logic (Muhammad Taha)

- Analyzed frequency of characters
- Constructed Huffman Tree using min-heap
- Assigned binary codes to characters
- Encoded text into compressed bitstream

Decompression & File I/O (Syed Shaheer Sohail)

- Added bit-level padding and byte array handling
- Created .huf file structure with header mapping
- Reconstructed Huffman Tree via decoding table
- Restored original file content

GUI Development (Syed Irfan Raza)

- Built Tkinter-based interface
- Buttons for file browsing, compress & decompress
- Real-time status updates
- Displayed compression statistics

Integration & Testing (Muhammad Abdullah)

- Combined all modules into one application
- Tested with multiple text files
- Added error handling (empty files, invalid inputs)
- Prepared documentation and final report

Compression Algorithm: Huffman Coding

Step 1: Frequency Calculation

The input file is scanned to count the frequency of each character. These frequencies are stored in a dictionary or frequency table.

Step 2: Build Min-Heap

Each character and its frequency are represented as a node. All nodes are inserted into a priority queue (min-heap), where the node with the smallest frequency has the highest priority.

Step 3: Construct Huffman Tree

The algorithm repeatedly extracts the two nodes with the lowest frequency and merges them into a new internal node whose frequency is the sum of both. This process continues until only one node remains the root of the Huffman Tree.

Step 4: Generate Huffman Codes

The tree is traversed from the root to each leaf:

- Moving left appends '0'
 - Moving right appends '1'
- This creates a prefix-free binary code for each character. The result is an encoding table.

Step 5: Encode and Store Output

Each character in the original file is replaced by its Huffman code, producing a compressed bitstring. This bitstring, along with padding info and the encoding table (header), is stored in a **.huf** file.

Step 6: Decompression

Decompression reverses this process using the same code mappings.

Advantages of Huffman Coding

- Lossless compression – no data is lost.
- Efficient for files with skewed frequency distributions.
- GUI enables easier usability for non-technical users.

Implementation

Compression Module

1. Frequency Calculation

Function: `get_frequency(text)`

- Counts occurrences of each character using a dictionary.

2. Huffman Tree Construction

Function: `build_huffman_tree(frequency)`

- Uses a min-heap to build the tree based on character frequencies.

3. Encoding Table Generation

Function: `build_encoding_table(root)`

- Traverses the Huffman Tree to assign binary codes.

4. Text Encoding

Function: `encode_text(text, encoding_table)`

- Converts the entire input text into a binary string.

5. Compression Workflow

Function: `compress(input_filepath, output_filepath)`

- Combines all steps: reads file → encodes → returns code + table.

Decompression Module

1. Padding Bit Handling

Function: `_pad_encoded_text(encoded_text)`

- Adds padding so bit length becomes divisible by 8.

2. Byte Conversion

Function: `_get_byte_array(padded_encoded_text)`

- Converts bitstream into bytearray for storage.

3. Writing Compressed File

Function: `write_compressed_file(output_filepath, encoded_string, encoding_table)`

- Saves padding, encoding table, and binary bytes into `.huf` file.

4. Padding Removal

Function: `_remove_padding(padded_encoded_text, padding_info)`

- Removes extra bits added earlier.

5. Decoding Bitstream

Function: `_decode_text(encoded_text, decoding_table)`

- Reads bits and converts them back into characters.

6. Full Decompression Workflow

Function: `decompress(input_filepath, output_filepath)`

- Extracts data → rebuilds text → writes original file.

GUI Integration

User Interaction Through Tkinter Widgets

The graphical interface is built using Tkinter components such as buttons, entry fields, labels, and text areas. These widgets allow users to select input files, specify output paths, and initiate compression or decompression operations without using the command line.

Event-Driven Function Calls

Buttons are linked to backend functions like `run_compression()` and `run_decompression()`, which trigger the Huffman encoding or decoding process. File dialogs are embedded using `filedialog.askopenfilename()` to smoothly pass selected file paths into the compression and decompression logic.

Real-Time Feedback and Results Display

The GUI updates users through status labels and message boxes for error handling. After

compression, statistical data such as original size, compressed size, and compression ratio are shown in a text area, ensuring users receive clear feedback on the tool's performance.

Time Complexity Analysis

Task	Time Complexity
Frequency Counting	$O(n)$
Heap Construction	$O(k)$
Tree Building	$O(k \log k)$
Encoding Text	$O(n)$
Decoding File	$O(n)$

Where:

- n = number of characters
- k = number of unique characters

Space Complexity Analysis

COMPONENT	SPACE
Frequency Table	$O(k)$
Huffman Tree	$O(k)$
Encoding Table	$O(K)$
Encoded Output	$O(n)$
Total Space Used	$O(n + k)$

Testing and Verification

The Huffman compression and decompression tool was subjected to functional end-to-end testing to verify the integrity of the algorithm, the file header structure, and the overall compression/decompression cycle.

The tests were performed on a large plain text file (football_50000_word.txt) to demonstrate performance and correctness on real-world data.

Functional Test 1: Compression Analysis

The first functional test verifies the system's ability to read an input file, build the Huffman tree, generate a compressed binary file (.huf), and calculate the resulting size reduction.

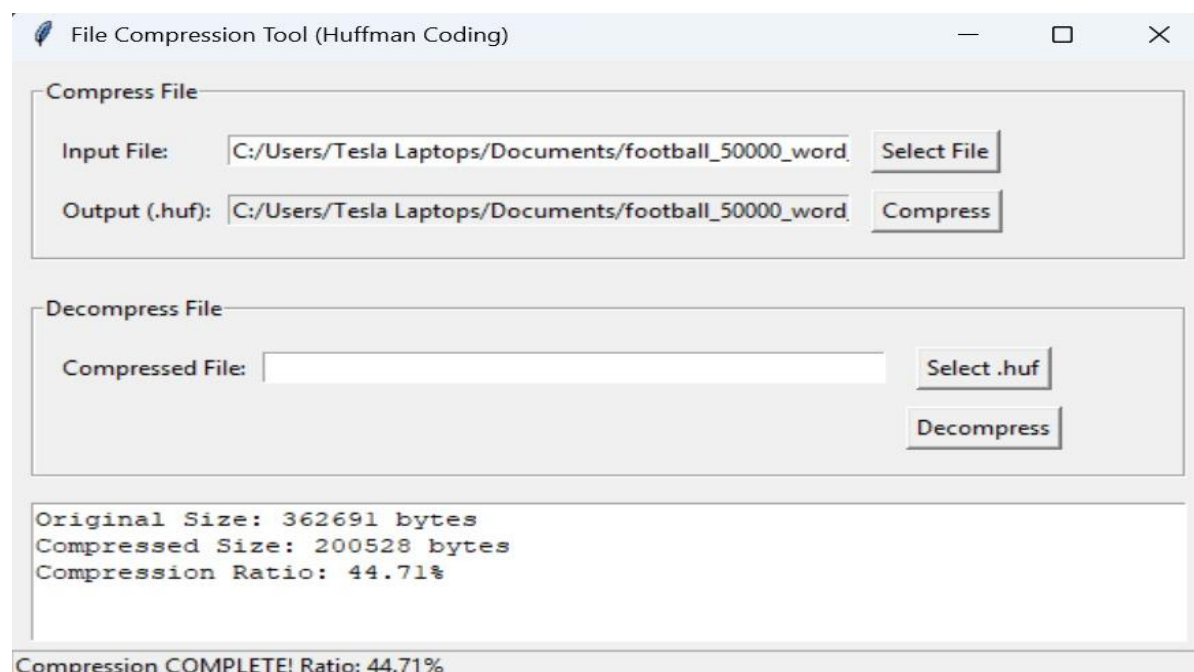
Process:

The input file (football_50000_word.txt) was selected.

The Compress function was executed.

The application saved the compressed binary data, including the Huffman code map and padding information, to the output file.

The results, as shown in the GUI output, confirm successful compression and significant data reduction:



Functional Test 2: Decompression and File Restoration

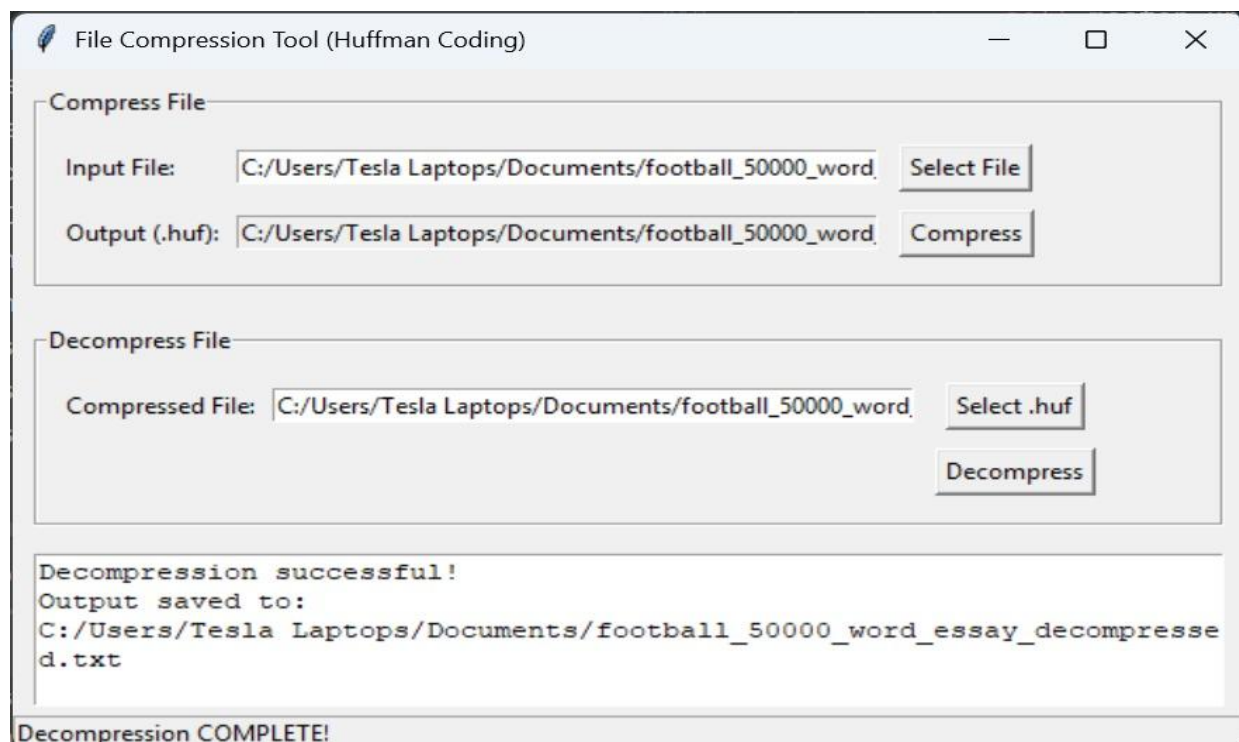
The second functional test ensures that the compressed data, including the header and padding, can be correctly read, decoded, and restored to its original state. This step is critical to proving the lossless nature of the compression.

Process:

The newly created compressed file (football_50000_word.huf) was selected for decompression.

The Decompress function was executed.

The system created a new output file named football_50000_word_essay_decompressed.txt



The output message, "Decompression successful!", confirms that the application correctly:

Read the header to obtain the decoding table.

Parsed the binary data.

Removed the padding bits.

Traversed the Huffman codes to reconstruct the original text content.

Lossless Verification

To formally conclude the test, the content of the original input file (football_50000_word.txt) was compared byte-for-byte with the content of the restored output file (football_50000_word_essay_decompressed.txt).

Result: The two files were found to be identical. This result is the definitive proof that the entire cycle of compression and decompression is lossless and that no data was corrupted or lost during the encoding and decoding process.

Conclusion:

This project successfully implements a Huffman Coding-based file compressor with a user-friendly GUI. The system is efficient, lossless, and shows real-world compression benefits. It demonstrates strong understanding of:

- Data encoding
- Tree data structures
- File handling
- GUI design
- Algorithmic complexity